

Project: Scalable Microservices Architecture with ECS and Fargate

Services: ECS, Fargate, ECR, RDS, CloudWatch

Description: Containerize a microservices-based application using Docker. Deploy using ECS Fargate and manage logs with CloudWatch. Use RDS as the backend database

Skills: Containers, microservices, monitoring.

Definition:

The Scalable Microservices Architecture with ECS and Fargate project focuses on deploying a containerized application using Docker and managing it through Amazon Elastic Container Service with AWS Fargate. The application container image is stored in Amazon Elastic Container Registry, allowing ECS to pull and run the container without managing servers. The system is designed to automatically scale and efficiently run microservices while monitoring logs and performance through Amazon CloudWatch, ensuring high availability, scalability, and simplified deployment of containerized applications in the cloud.

Scope of the Project:

- Containerization of Application-The project involves packaging the application into containers using Docker to ensure portability and consistent runtime environments.
- Container Image Management-The container images are stored and managed in Amazon Elastic Container Registry so that they can be securely accessed and deployed.
- Container Orchestration-The deployment and management of containers are handled by Amazon Elastic Container Service, which schedules and runs the container tasks.
- Serverless Container Execution-The containers run on AWS Fargate, eliminating the need to manage underlying servers or infrastructure.
- Monitoring and Logging-Application logs and performance metrics are collected and monitored using Amazon CloudWatch for troubleshooting and system monitoring.
- Scalable Microservices Deployment-The project demonstrates how microservices can be deployed in a scalable and reliable cloud environment to support modern application architectures.

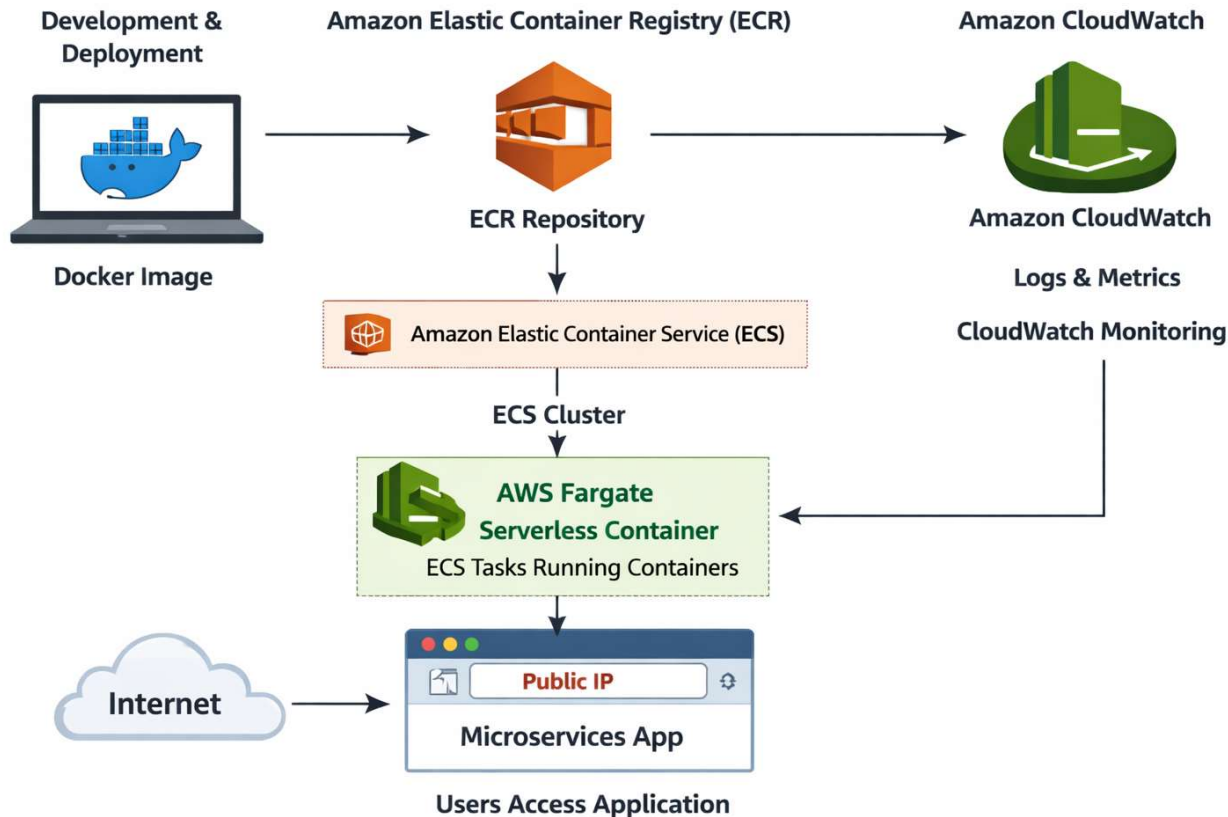
Methodology:

1.EC2-Based Container Deployment

- Create an EC2 instance using Amazon EC2.
- Install Docker on the EC2 instance to build and run container images.
- Build the application container image and push it to Amazon Elastic Container Registry.
- Create a cluster in Amazon Elastic Container Service using EC2 launch type.
- Deploy the containerized application on ECS using EC2 instances as the underlying infrastructure.
- Monitor container logs and performance using Amazon CloudWatch.

2.Serverless Container Deployment Using Fargate

- Containerize the application using Docker.
- Store the container image in Amazon Elastic Container Registry.
- Create a cluster in Amazon Elastic Container Service. Define a task definition that specifies container configuration and resources.
- Deploy the application using AWS Fargate to run containers without managing servers.
- Monitor application logs and metrics using Amazon CloudWatch.

Architecture Diagram:**AWS Services used in the Project:****1. Amazon Elastic Container Service**

- **Definition:** Amazon ECS is a fully managed container orchestration service that helps deploy, manage, and scale containerized applications.
- **Role in the Project:** It manages the deployment and scheduling of containerized microservices within the ECS cluster.
- **Purpose:** To orchestrate and manage containers running the application.

2. AWS Fargate

- **Definition:** AWS Fargate is a serverless compute engine that allows users to run containers without managing servers or infrastructure.
- **Role in the Project:** It runs the ECS tasks and executes the containerized application.
- **Purpose:** To provide serverless infrastructure for running containers efficiently.

3. Amazon Elastic Container Registry

- **Definition:** Amazon ECR is a fully managed container image registry used to store, manage, and retrieve Docker container images.
- **Role in the Project:** It stores the Docker image of the microservices application used by ECS for deployment.
- **Purpose:** To securely store and manage container images required for the application.

4.Amazon CloudWatch

- **Definition:** Amazon CloudWatch is a monitoring service that collects logs, metrics, and events from AWS resources and applications.
- **Role in the Project:** It monitors container logs and application performance generated by ECS tasks.
- **Purpose:** To track system performance and help in troubleshooting issues.

5.Amazon EC2

- **Definition:** Amazon EC2 is a cloud computing service that provides scalable virtual servers for running applications.
- **Role in the Project:** It is used to build and test the Docker container image before pushing it to ECR.
- **Purpose:** To provide a compute environment for developing and building container images.

Project Implementation Steps:

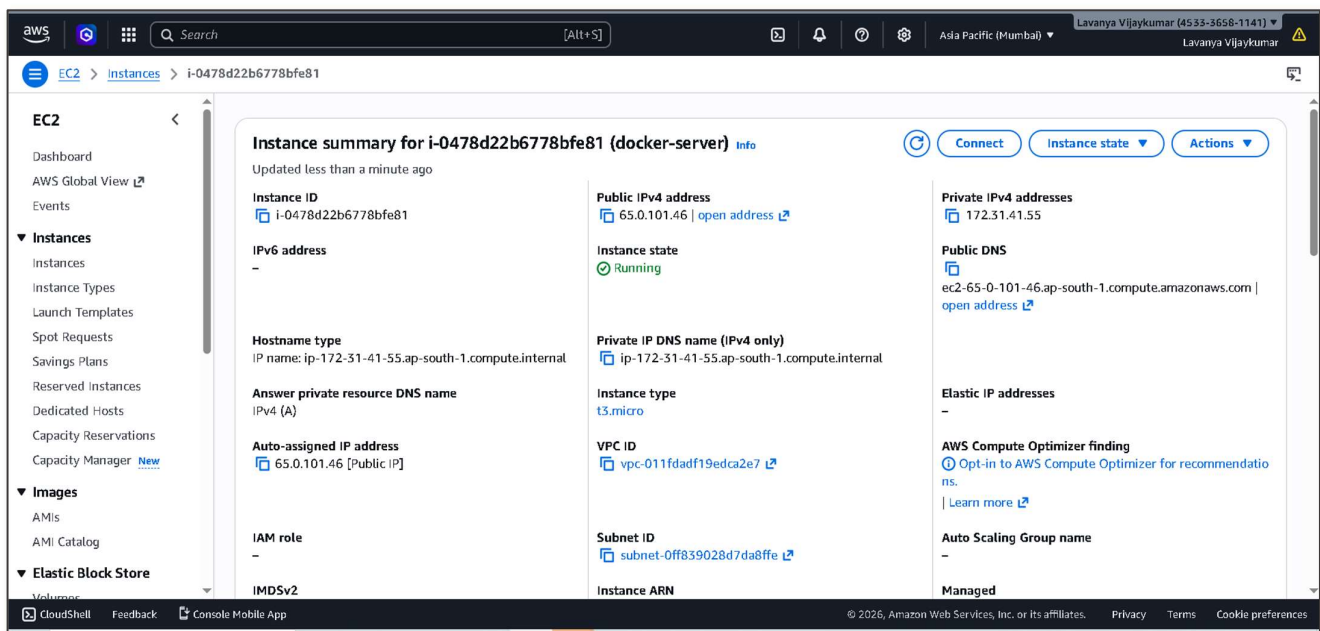
STEP 1:Launch an EC2 Instance

- Click Launch Instance.
- Choose Ubuntu Server as the operating system.
- Configure security group to allow:

SSH (Port 22)

HTTP (Port 80)

- Launch the instance



STEP 2: Install Docker on EC2

```
*** System restart required ***
Last login: Thu Mar 12 02:27:57 2026 from 13.233.177.3
ubuntu@ip-172-31-41-55:~$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~22.04.1
ubuntu@ip-172-31-41-55:~$
```

STEP 3: Create Application Files

- Create project directory:
mkdir microservice-app
cd microservice-app
- Create HTML file:
nano index.html
Add content:
- Save and exit.

```
ubuntu@ip-172-31-41-55:~$ cd microservice-app/
ubuntu@ip-172-31-41-55:~/microservice-app$ ls
Dockerfile  index.html
ubuntu@ip-172-31-41-55:~/microservice-app$
```

STEP 4: Create Dockerfile

- Create Dockerfile:
nano Dockerfile
- Save and exit.

```
ubuntu@ip-172-31-41-55:~$ cd microservice-app/
ubuntu@ip-172-31-41-55:~/microservice-app$ ls
Dockerfile  index.html
ubuntu@ip-172-31-41-55:~/microservice-app$
```

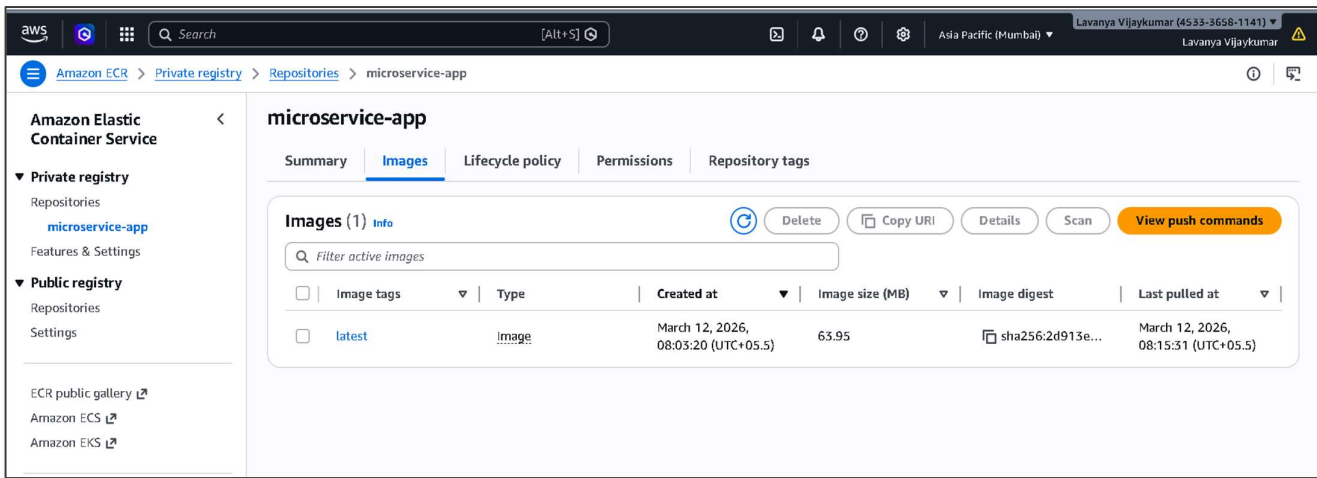
STEP 5: Build Docker Image

- Build container image:
- sudo docker images

```
ubuntu@ip-172-31-41-55:~/microservice-app$ ls
Dockerfile  index.html
ubuntu@ip-172-31-41-55:~/microservice-app$ docker images
REPOSITORY                                TAG      IMAGE ID      CREATED        SIZE
453336581141.dkr.ecr.ap-south-1.amazonaws.com/microservice-app  latest   0c83fa6c1f0e  3 hours ago   161MB
453336581141.dkr.ecr.us-east-1.amazonaws.com/microservice-app    latest   0c83fa6c1f0e  3 hours ago   161MB
microservice-app                                                  latest   0c83fa6c1f0e  3 hours ago   161MB
nginx                                                            latest   99133eed2307  31 hours ago  161MB
ubuntu@ip-172-31-41-55:~/microservice-app$
```

STEP 6: Create Container Repository

- Click Create Repository
- Repository name: microservice-app
- Click Create Repository



STEP 7: Authenticate Docker with ECR

- `aws ecr get-login-password --region ap-south-1 | sudo docker login --username AWS --password-stdin <account-id>.dkr.ecr.ap-south-1.amazonaws.com`
- Login will be successful.

```
ubuntu@ip-172-31-41-55:~/microservice-app$ ls
Dockerfile  index.html
ubuntu@ip-172-31-41-55:~/microservice-app$ docker images
REPOSITORY                                     TAG          IMAGE ID          CREATED          SIZE
453336581141.dkr.ecr.ap-south-1.amazonaws.com/microservice-app  latest       0c83fa6c1f0e      3 hours ago     161MB
453336581141.dkr.ecr.us-east-1.amazonaws.com/microservice-app    latest       0c83fa6c1f0e      3 hours ago     161MB
microservice-app                                                  latest       0c83fa6c1f0e      3 hours ago     161MB
nginx                                                            latest       99133eed2307      31 hours ago    161MB
ubuntu@ip-172-31-41-55:~/microservice-app$
```

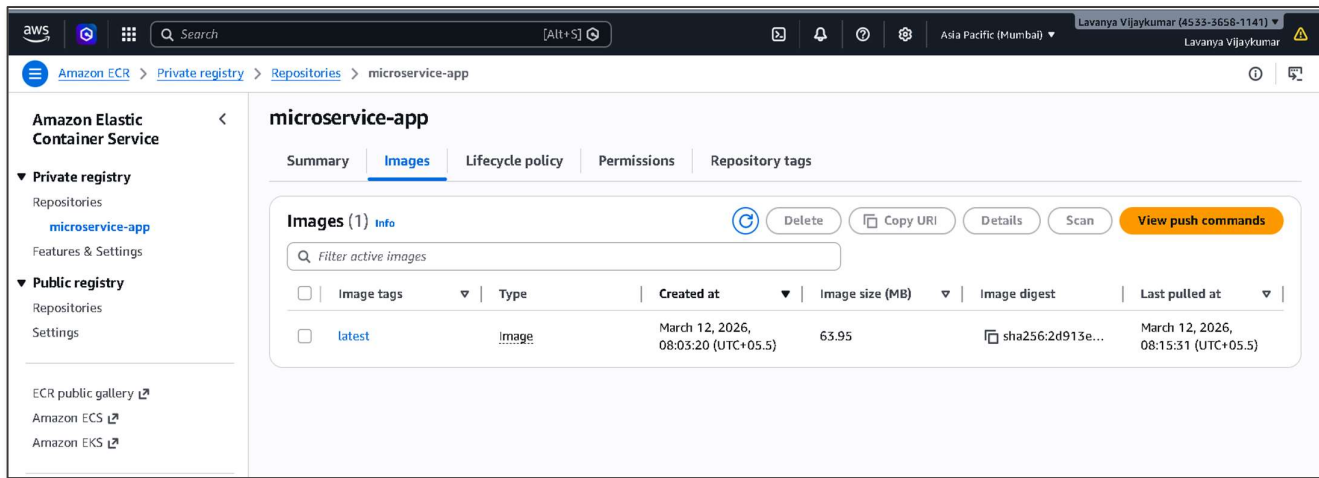
STEP 8: Tag Docker Image

- Tag the image to push to ECR.
- `sudo docker tag microservice-app:latest <account-id>.dkr.ecr.ap-south-1.amazonaws.com/microservice-app`

```
ubuntu@ip-172-31-41-55:~/microservice-app$ ls
Dockerfile  index.html
ubuntu@ip-172-31-41-55:~/microservice-app$ docker images
REPOSITORY                                     TAG          IMAGE ID          CREATED          SIZE
453336581141.dkr.ecr.ap-south-1.amazonaws.com/microservice-app  latest       0c83fa6c1f0e      3 hours ago     161MB
453336581141.dkr.ecr.us-east-1.amazonaws.com/microservice-app    latest       0c83fa6c1f0e      3 hours ago     161MB
microservice-app                                                  latest       0c83fa6c1f0e      3 hours ago     161MB
nginx                                                            latest       99133eed2307      31 hours ago    161MB
ubuntu@ip-172-31-41-55:~/microservice-app$
```

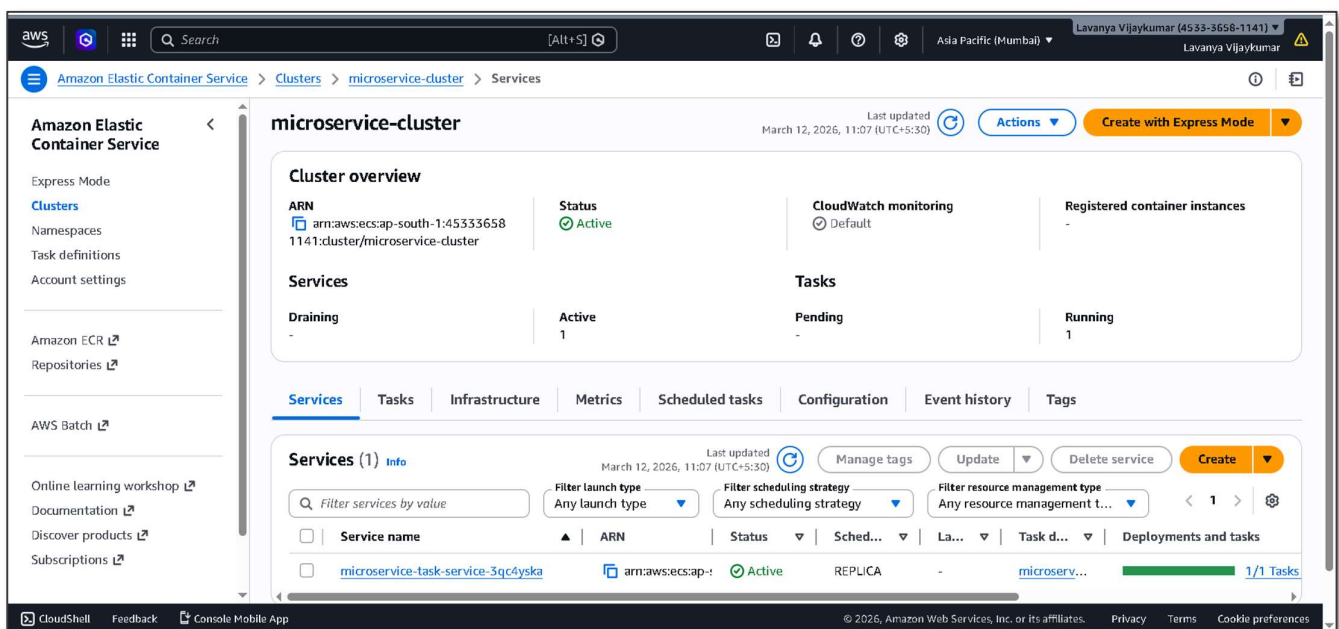
STEP 9: Push Image to ECR

- Push container image:
- `sudo docker push <account-id>.dkr.ecr.ap-south-1.amazonaws.com/microservice-app`



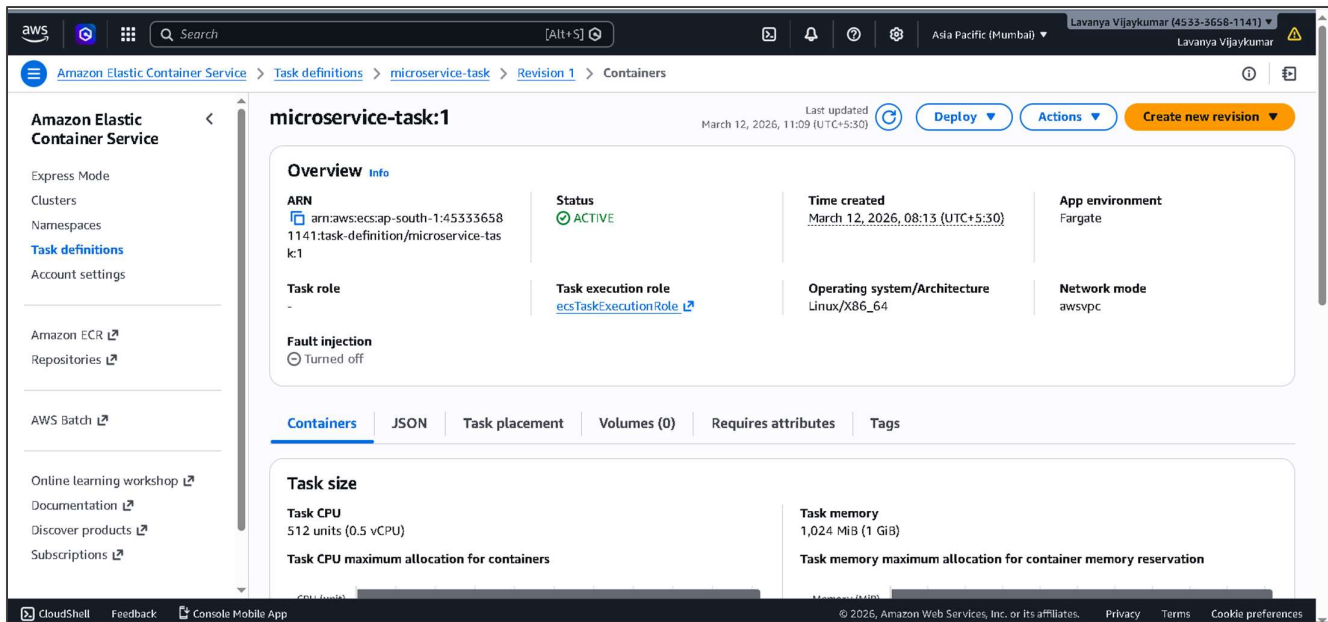
STEP 10: Create ECS Cluster

- Click Clusters
- Select Networking only (Fargate).
- Cluster name: microservice-cluster
- Click Create.



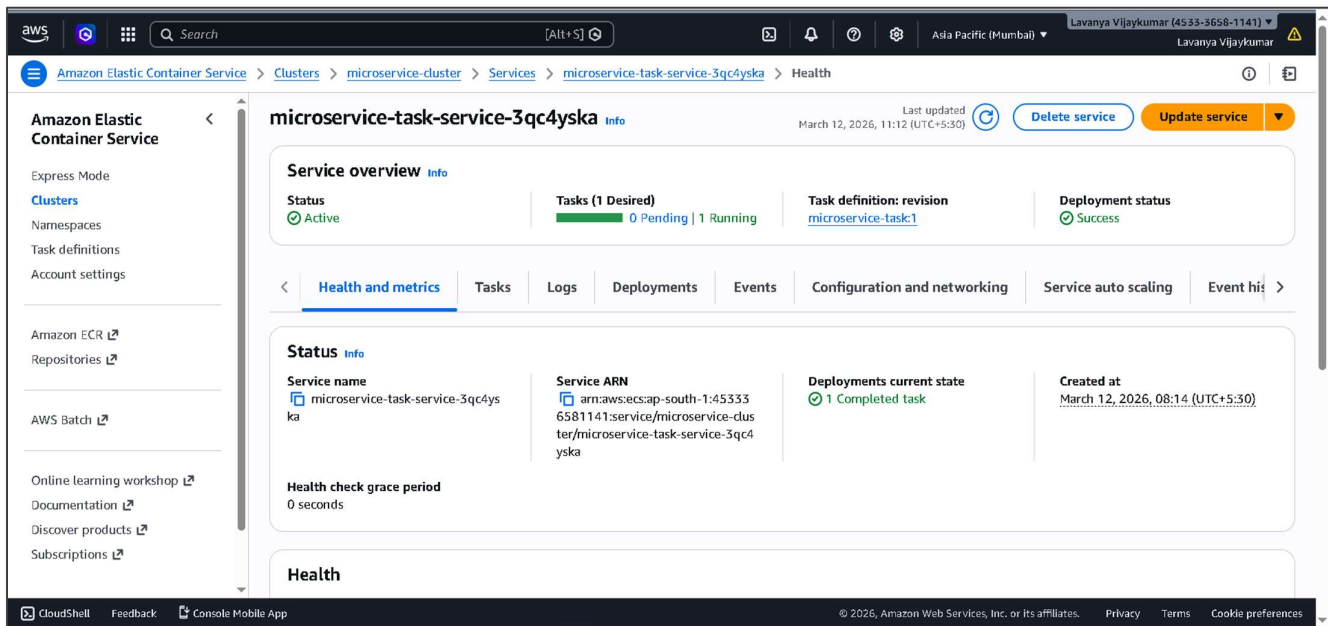
STEP 11: Create Task Definition

- Go to Task Definitions
- Click Create Task Definition
- Select Fargate
- Task Name: microservice-task
- Container name: web-container
- Image: <account-id>.dkr.ecr.ap-south-1.amazonaws.com/microservice-app:latest
- Port mapping: 80
- Save the task definition.



STEP 12: Create ECS Service

- Open the cluster.
- Click Create Service.
- Task Definition:microservice-task
- Number of tasks:1
- Enable public IP.
- Click Create Service.



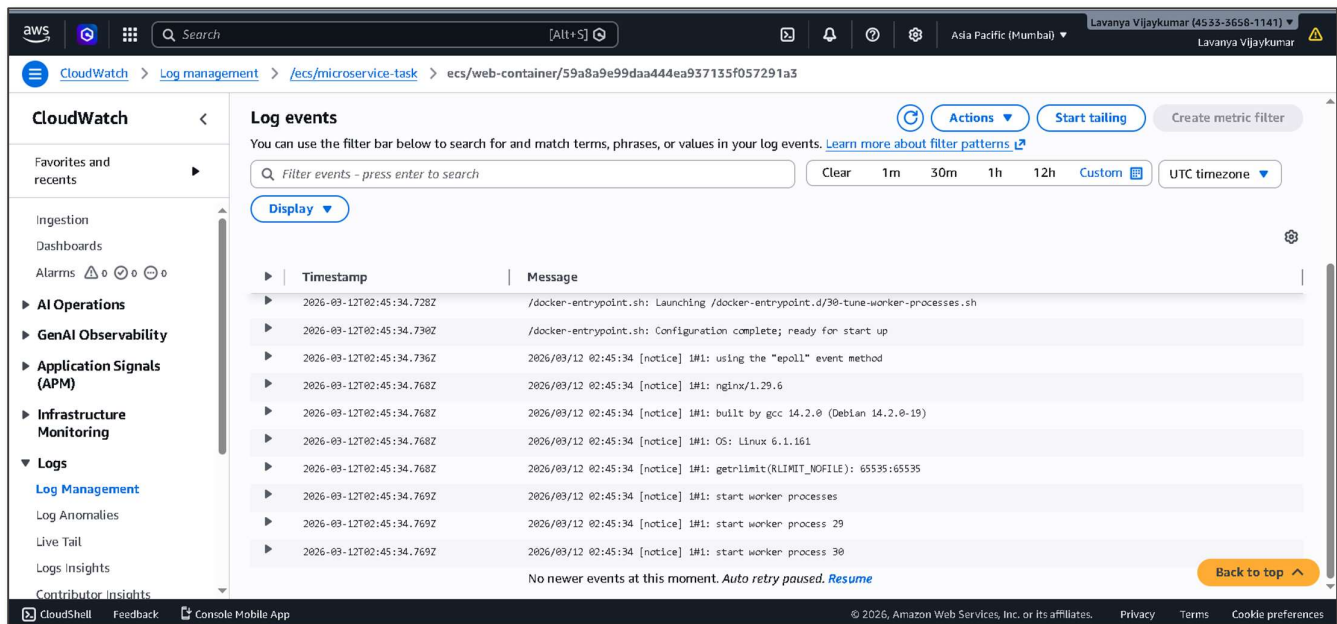
STEP 13: Access the Application

- Open the task.
- Go to Networking section.
- Copy the Public IP.
- Open browser:http://Public-IP
- The webpage will display



STEP 14: Monitor Logs

- Go to:
- Logs → Log Groups
- Open the log group created for ECS
- Will be able to see container logs and metrics.



Troubleshooting:

Issue :Unable to push Docker image to ECR

- **Cause:** The Docker image was not tagged correctly or the repository does not exist.
- **Solution:** Create a repository in Amazon Elastic Container Registry and tag the image correctly before pushing it.

Conclusion:

The Scalable Microservices Architecture with ECS and Fargate project demonstrates how containerized applications can be efficiently deployed and managed in a cloud environment. By using Docker for containerization and storing images in Amazon Elastic Container Registry, the application can be easily deployed and maintained. The containers are orchestrated using Amazon Elastic Container Service and executed through AWS Fargate, eliminating the need to manage servers. Additionally, monitoring and logging through Amazon CloudWatch ensures better visibility and troubleshooting. Overall, the project highlights a scalable, reliable, and serverless approach for deploying modern microservices applications on AWS.